NexusOS

A multi-tenant business operating platform built security-first: authentication, authorization and tenant isolation solved as platform infrastructure before any product module was allowed to depend on them.

Overview

NexusOS is a web platform designed to host multiple business capabilities — CRM, projects, finance, HR, inventory, documents — behind one consistent application shell, one identity model and one tenant boundary.

The premise is that business software fragments because each capability arrives with its own idea of who the user is, which company they belong to and what they are allowed to do. NexusOS answers those three questions once, at the platform layer, so that every module added later inherits the same identity, the same permission model and the same isolation guarantees rather than reimplementing them.

The current build delivers that platform layer end to end. The product modules that will sit on top of it are, with one exception, not yet built — and this case study is explicit about where that line falls.

The Challenge

The straightforward way to build a product like this is to ship a CRM first, add projects next, and retrofit multi-tenancy and permissions once there are customers. That path is faster to a demo and structurally very hard to recover from. Tenant scoping becomes a filter that each query is individually responsible for remembering, and a single forgotten `WHERE company_id = ...` becomes a cross-tenant data leak.

The harder and more defensible sequence is to treat identity, authority and isolation as infrastructure — decided once, enforced centrally, and impossible for a feature author to skip.

That reordering created the actual engineering problems this project had to solve:

Tenant identity cannot be an input. If any part of an authenticated request can influence which company's data it reads — a header, a path segment, a token claim — then tenant isolation is only as strong as the discipline of whoever writes the next endpoint. Tenant identity has to be derived, not accepted.

A valid token is not a valid authorization. A signed JWT proves only that the token was issued. Between issuance and expiry, a user can be suspended, stripped of a role, or have their whole company deactivated. If permissions are read from token claims, every one of those actions has a silent delay before it takes effect.

Email is not a unique identity in a multi-tenant system. The same person can legitimately hold accounts at two different companies, which means the login lookup itself needs tenant context before it can identify anyone.

A design system is infrastructure too. Ten product modules built as independently styled pages produce ten divergent interpretations of a table, a form error and an empty state. The component and token layer had to exist before the modules, for the same reason the auth layer did.

My Role

Sole engineer. NexusOS has no other human contributors — I owned system architecture, backend, frontend, database design, security engineering, the design system, the test suite and the local/CI build pipeline.

In practice that meant deciding the layering (routers stay thin, services hold logic, dependencies own identity), designing the tenant data model and its constraint strategy, implementing the authentication and RBAC layers and the middleware chain around them, building the component and token system the interface is assembled from, and writing the test suite — including the security tests that assert the isolation and fail-closed behaviour the architecture depends on.

Architecture

NexusOS is a single-page React client talking to a FastAPI service over a versioned REST API, with PostgreSQL as the system of record.

Frontend — React 19 and TypeScript on Vite 6, with route-level code splitting and vendor chunking. Routing is a declarative table with an authenticated shell behind a route guard and permission-gated branches. Client state is split deliberately: a small persisted store holds session state, while server data lives in a query cache with its own staleness rules.

Backend — FastAPI built through an application factory, with a middleware chain, routers and exception handlers wired in one place. Layering is enforced by convention and is consistent across the codebase: routers validate and delegate, services hold business logic, and identity is resolved by dependencies rather than by each endpoint. Every response — success or failure — is returned in the same envelope, so the client has exactly one shape to handle.

Data — PostgreSQL 16 accessed asynchronously through SQLAlchemy 2.0 over `asyncpg`, with a connection pool and liveness checks. Schema changes go through Alembic, and CI runs the migrations before the tests to catch drift between models and migration history. Primary keys are UUID v7, which keeps identifiers non-sequential without giving up index locality.

Redis serves one purpose: shared counters for authentication rate limiting. It is not a session store and not a cache. Naming it accurately matters — a diagram that shows Redis as a generic caching layer would misrepresent the system.

Sessions and transactions are request-scoped. One database session per request, committed on success and rolled back on any raised exception, so a handler cannot leave a partial write behind.

The full component map is in the **System Architecture** artifact.

Multi-Tenant Architecture

This is the core of the system, and the part worth reading the code for.

Where each identity comes from

A company is the tenant. Users, roles and business records all carry a `company_id`, and users are unique by email *within* a company rather than globally — the same person can hold accounts in two tenants without collision.

At sign-in, the client supplies a company slug alongside email and password. This is the one and only point where the client influences tenant selection, and it does so the way a subdomain would: it says which tenant to authenticate *against*. It does not grant anything. The credentials still have to be correct for a user inside that company.

After sign-in, tenant identity stops being an input:

Identity	Asserted by the client	Authoritative source
User	Token subject claim	Database row, which must exist and be active
Tenant / company	Present in the token — and ignored	Read from the stored user record
Role	Present in the token — and ignored	Re-read from the database per request
Permissions	Never carried in a token	Flattened from role grants at request time
Session	Refresh token identifier	Matched against a live server-side session record

The access token does carry `company_id` and `role`. Neither is trusted. When a protected request arrives, the token's subject claim is used only to locate the user row; every value that an authorization or scoping decision depends on is then read from that row and its relationships. The dependency that does this carries the comment `# from the DB row, never the token claim`, and there is a test asserting exactly that behaviour.

The practical consequence: **an authenticated request cannot influence its own tenant scope.** There is no tenant header, path segment or body field on authenticated routes. A valid token for one company cannot be pointed at another, because nothing in the request is consulted to decide which company it belongs to.

What isolation is actually enforced by

Two mechanisms, and it is worth being precise about which is which.

Application-level scoping. Tenant-aware queries are scoped by the server-derived `company_id`. A tenant-scoped repository foundation implements this generically — it refuses to build a query for a tenant-owned model when no company is set, so the failure mode of forgetting tenant context is an exception rather than a silent full-table read. That foundation is built and tested but is not yet wired into live routes, since the CRUD endpoints it exists to serve are not built. The authentication service, which is live, does its tenant scoping through explicit queries.

Relational constraints. Child tables reference their parents by composite foreign key — `(parent_id, company_id)` against `(id, company_id)` — so the database itself rejects a row that would attach a record in one tenant to a parent in another. This holds regardless of application

code correctness, and there are tests that attempt cross-tenant references and assert the database refuses them.

NexusOS does not use PostgreSQL row-level security. No policies exist and no per-request session variable is set. Isolation is enforced in the application layer and by foreign-key constraints, as described above. RLS would be a meaningful addition — it would make isolation independent of the application entirely — but claiming it here would be false, and the distinction between "the application always scopes correctly" and "the database will not return the row regardless" is exactly the kind of thing a reviewer should be able to trust a case study about.

The request lifecycle is traced end to end in the **Multi-Tenant Request Flow** artifact.

Authentication

Registration creates the tenant and its first user in one flow: a company record with a unique slug, the five default roles with their permission grants, the owner user, and an issued session. An email verification token is generated at the same time.

Login joins user and company filtered by company slug, email and active status, verifies the password, rejects suspended accounts, records the sign-in and issues a token pair. Failures return one generic message regardless of whether the tenant, the user or the password was wrong, so the endpoint cannot be used to enumerate accounts.

Access tokens are short-lived — fifteen minutes — and carry subject, type, issued-at, expiry, a unique token ID, plus company and role for informational purposes. They are signed with a pinned algorithm, and decoding validates the token's declared type, so a refresh token cannot be presented where an access token is expected.

Refresh tokens last thirty days and are rotated on every use. The raw token is never stored; the server keeps a session record holding the token's unique identifier, its expiry and its revocation timestamp. Refreshing revokes the old session and issues a new pair.

Reuse detection is the part I'd point a reviewer at. Presenting a refresh token that has already been spent is treated as evidence of theft: every session belonging to that user is revoked, and the event is written to the audit log. A stolen refresh token therefore buys an attacker one use before locking out both them and the legitimate user, which surfaces the compromise instead of hiding it.

Logout revokes the presented session and is idempotent. **Password reset** consumes a single-use, hash-stored opaque token and revokes every session for that user. **Email verification** uses the same single-use token mechanism.

Passwords are hashed with **Argon2**, a memory-hard function, via passlib. Verification and reset tokens are deliberately *not* JWTs — they are high-entropy random values stored as hashes, precisely so they can be invalidated server-side before their expiry, which a stateless token cannot be.

Two honest limitations: email delivery is a logging stub rather than a wired provider, and sign-in does not currently require a verified email address.

Authorization

Roles are per-tenant database rows, not an enum. Registration seeds five — Owner, Admin, Manager, Employee, Viewer — and grants each a set of permissions from a global catalogue of `module.action` strings. Wildcards are expanded once, at grant time, so permission checks at request time are plain set operations with no pattern matching and no implicit matches.

Evaluation is where the design earns its keep. Two dependencies exist, and the split is deliberate:

- One decodes and validates the token. It is cheap, hits no database, and proves only that the token is validly signed and unexpired. Its role field is a claim, and may be stale by up to the token's lifetime.
- The other is the authoritative one used for anything that makes an authorization decision. It loads the user with roles, permissions and company, and **fails closed** if the user is missing, soft-deleted, inactive or suspended, or if their company is deactivated or deleted.

Route guards for roles and permissions both build on the authoritative dependency, so every guard inherits the fresh-from-database check automatically. Permission checks require every listed permission, and anything missing or unrecognised denies.

The reason token claims are not trusted for authority is concrete: it closes the stale-privilege window. Suspending a user or revoking a permission takes effect on their **next request**, not whenever their current access token happens to expire. Twelve tests cover this specifically, including that permissions are hydrated from the database rather than the token, that a stale token role loses to the current database role, and that suspended users, deactivated users and deactivated companies each fail closed.

Security Engineering

Controls verified as implemented and wired. The **Security Architecture** artifact lays these out as the layers a request passes through.

Request boundary. Cross-origin access runs against an explicit allowlist with credentials enabled. Startup validation refuses to boot in production on a wildcard origin, an empty origin list, or a plaintext HTTP origin outside localhost — the misconfiguration fails the deploy instead of silently weakening the running service. Every response carries baseline headers: content-type sniffing disabled, framing denied, no-referrer, no-store. HSTS is emitted when the request is HTTPS or when explicitly enabled, which handles TLS terminating at an edge the application cannot see.

Client IP resolution. Forwarded headers are ignored by default and honoured only when the immediate peer falls inside a configured trusted network, with caps on hop count and header size, and universal CIDR ranges rejected at startup. One resolver feeds rate limiting, transport-security decisions and the audit trail, so a spoofed header cannot desynchronise them. This is a small piece of code that gets skipped more often than it should — an IP-keyed rate limiter reading `X-Forwarded-For` without a trust boundary is trivially bypassed.

Rate limiting. Six credential-accepting endpoints are limited per resolved client address using shared counters, so limits hold across application instances rather than per process. The

limiter fails open if its backing store is unreachable — a deliberate availability trade-off, and one worth stating rather than glossing.

Request context. Every request gets a sanitised identifier — validated against a strict character and length rule so a client-supplied value cannot inject into logs — bound to structured logs and echoed back on the response. The unhandled-exception path re-applies the identifier and security headers directly, because that handler runs outside the middleware chain and would otherwise return a response missing both.

Audit trail. Append-only records capturing actor, tenant, action, entity, resolved IP and user agent, currently wired for registration, successful sign-in and refresh-token reuse.

Log redaction. Credential-bearing keys are scrubbed from structured logs and from audit payloads. The mechanism is key-based, so it does not scan free text — a known limitation, documented in the code rather than assumed away.

Error handling. Domain errors map to a uniform envelope with a stable code. Unhandled exceptions return a generic message with no internals or stack traces, while the full trace goes to the server-side log. API documentation endpoints are disabled in production.

Data access. All queries go through the ORM with parameter binding. The only raw SQL in the application is the readiness probe's static `SELECT 1`.

Secrets. Loaded from the environment with a minimum signing-key length enforced and known placeholder values rejected in production. No production secrets are committed.

Deliberately not claimed

Stating what is absent is part of the point:

- **PostgreSQL row-level security** — no policies exist; isolation is application-level and relational.
- **Content-Security-Policy and Permissions-Policy** — not set; deferred to the static host for a JSON API.
- **JWT issuer and audience claims** — not issued, not validated.
- **Access-token revocation before expiry** — tokens are stateless for their fifteen-minute life; the mitigation is per-request database re-validation of account and tenant status.
- **Account lockout after repeated failures** — rate limiting only.
- **Verified-email enforcement at sign-in** — verification exists but is not required to authenticate.

Frontend Engineering

The frontend is built as a system rather than a set of pages.

Routing and guards. A route table splits a public authentication shell from an authenticated application shell behind a guard, with permission-gated branches that render an explicit forbidden state rather than a misleading 404. Pages are lazily loaded behind a suspense boundary with its own error boundary, and a prefetch registry warms routes ahead of navigation.

State. Two stores with distinct jobs. Session state lives in a small persisted store; server data lives in a query cache with its own staleness policy and is cleared wholesale when a session ends. Keeping these separate avoids the common failure where cached tenant data survives a user switch.

API client. A single client attaches the bearer header, unwraps the response envelope into typed data or throws a typed error, and handles expiry. The refresh path is worth calling out: a 401 triggers at most one retry per request, and concurrent 401s share a **single in-flight refresh** rather than each firing their own — without that, a dashboard issuing six parallel requests would trigger six refreshes, and token rotation would invalidate five of them. A failed refresh clears auth state, clears the query cache and redirects to sign-in with an explicit expiry signal.

Design system. 157 CSS custom properties define a surface ladder, chart palette, radii, elevation and typography, mapped into Tailwind so components consume tokens rather than raw values. 31 components are built on Radix primitives with variant-driven styling. Theme switching is light, dark or system, persisted locally.

Forms. React Hook Form with Zod resolvers, so validation rules are declared once as a schema and shared between the form and the typed request payload.

System states. Loading skeletons, empty states, error states with recovery actions, offline detection and toasts exist as shared components, which is what makes them consistent across modules that haven't been written yet.

Interaction surfaces. A command palette, a global capture dialog driven by an action registry, a notifications drawer and an account dialog are wired into the shell.

Product Interface

The screenshots show the application shell and the design system, and it is worth being precise about what is behind each one.

Backend-connected. The authentication screens — sign-in, registration, forgotten password, reset, email verification — run against the live API. Sign-in's "Company URL" field is the tenant selector described in the multi-tenancy section, not decoration. The account dialog reflects real session state.

Interface-complete, rendering fixture data. The dashboard is fully built as an interface — KPI grid with sparklines, revenue and expense charts, pipeline by stage, activity feed, upcoming meetings and an insights panel — and is served by a fixture module standing in for an aggregate endpoint that does not exist yet. **The insights panel is a UI component; there is no AI integration in the backend.** Configuration reserves a key for a provider, and conversation and report tables are modelled, but no inference code exists. The screenshots demonstrate interface and data-visualisation engineering, not a working analytics or AI pipeline.

Design system documentation. The engineering screenshots come from a living style-guide route rendering the real token and component system — surfaces, typography, buttons, form controls with live validation, feedback states, and dashboard primitives. These are the components the product modules will be assembled from.

Navigation only. CRM, projects, tasks, HR, finance, inventory, documents, reports, assistant and settings appear in the sidebar and route to an explicit placeholder state.

Testing & Reliability

Backend — 167 test functions across 20 files, run under pytest against real PostgreSQL and Redis services in CI rather than mocks. 111 of those sit in a dedicated security suite:

- **Authorization (12)** — permissions hydrated from database not token; suspended, deactivated and soft-deleted users each fail closed; deactivated and deleted companies fail closed for their members; deny-by-default; a stale token role losing to the current database role.
- **Tenant isolation (16)** — one tenant's repository cannot see another's rows; cross-tenant fetch returns not found; querying a tenant model with no company set fails closed; the same email is allowed in two companies; cross-tenant update, restore, delete, count, exists and bulk-fetch all rejected; server-side pagination clamping and a sort-field allowlist.
- **Cross-tenant relationships (4)** — the database rejects contacts, leads and invoices referencing another tenant's customer.
- **Trusted proxy (43 across three files)** — IP and scheme resolution, spoof resistance, CIDR validation, and end-to-end behaviour through the real ASGI application.
- **Plus** request-ID handling, redaction, exception logging, email-log safety, session expiry, mass-assignment rejection and rate limiting.

Frontend — 89 test cases across 21 files under Vitest with Testing Library, covering the authentication pages, the auth service and validation schemas, and the UI primitives.

CI — every push and pull request runs backend lint, static type checking, Alembic migrations plus a drift check, seeding and pytest; and frontend type checking, tests and a production build. Migrations running before tests means a model that has drifted from its migration history fails the build rather than a deploy.

Counts are test functions defined in the repository. Coverage is deliberately weighted toward security and tenant behaviour; the dashboard interface and shell are not meaningfully covered.

Engineering Decisions

1 — Authorization re-derived from the database, not read from token claims

Why: A JWT proves issuance, not current authority. Reading role and permissions from claims means suspension and permission revocation take effect only when the token expires.
Trade-off: One indexed query with two eager-loaded relationships on every protected request. That cost buys immediate revocation, and it is the reason access tokens can safely be stateless for their lifetime.

2 — Tenant scope derived server-side, never accepted from the request

Why: If any request field can select a tenant, isolation depends on every future endpoint author remembering to validate it. Deriving it from the stored user record makes the safe path the only path.

Trade-off: No native support for a user belonging to multiple tenants in one session — switching tenants requires re-authenticating against that company. Correctness over convenience, and reversible later behind an explicit, validated switch.

3 — Short-lived access tokens with rotating refresh sessions and reuse detection

Why: Stateless tokens scale well but cannot be revoked; server-side sessions revoke cleanly but cost a lookup. Splitting the roles keeps request-path validation cheap while retaining real revocation, and treating a replayed refresh token as theft turns a silent compromise into a visible, self-limiting event.
Trade-off: More moving parts than a single long-lived token — rotation, session records, and a client that handles concurrent refreshes correctly.

4 — Tenant isolation enforced by composite foreign keys, not only by application code

Why: Application-level scoping is one bug away from failing. A composite key referencing `(id, company_id)` makes a cross-tenant relationship unrepresentable in the database.
Trade-off: Wider keys and indexes, and a denormalised `company_id` on child tables. Cheap relative to a cross-tenant leak.

5 — Design tokens and shared primitives before product modules

Why: Ten modules built independently produce ten interpretations of a table, an error and an empty state. Building the token layer and component library first means every module inherits consistency rather than negotiating it.
Trade-off: Meaningful time spent on a style-guide route and component tests before shipping a single product feature.

6 — Domain schema modelled ahead of the product surface

Why: Modelling CRM, projects, finance, HR and inventory together forced the tenant column strategy, soft-delete policy, audit columns and cross-entity constraints to be decided once, consistently, rather than five times with drift.
Trade-off: 38 model classes exist for which no endpoints are built yet — schema that will need revision as real endpoints expose real requirements. This is the most debatable decision here, and the reason the tenant standard is uniform across every table.

Current State

Implemented

- Multi-tenant identity model: companies, users, per-tenant roles, global permission catalogue, sessions, audit log
- Full authentication lifecycle — registration with tenant bootstrap, login, refresh with rotation and reuse detection, logout, password reset, email verification
- RBAC with database-revalidated role and permission evaluation, deny-by-default guards, fail-closed on suspended users and deactivated tenants
- Security layer — CORS with production validation, security headers, rate limiting, trusted-proxy client IP resolution, request context, audit logging, redaction, uniform error envelope, startup configuration validation

- Frontend platform — routing with guards, session and server-state stores, API client with single-flight refresh, 31-component design system on 157 tokens, form and validation stack, system-state components, command palette and capture surfaces
- 167 backend and 89 frontend test functions, with CI running lint, type checks, migrations and both suites

Foundation implemented — schema or interface prepared

- Domain schema for CRM, projects, tasks, HR, finance, inventory, documents, notifications and workflows: 38 model classes migrated, with tenant columns, audit columns, soft delete and cross-tenant constraints — no endpoints yet
- Tenant-scoped repository with mass-assignment protection, pagination clamping and sort allowlisting: implemented and tested, not yet wired to live routes
- Dashboard: interface complete, served by fixture data pending an aggregate endpoint
- Email delivery: service boundary and token lifecycle complete, provider not wired

Planned — not implemented

- All product modules behind the sidebar: CRM, projects, tasks, HR, finance, inventory, documents, reports, settings
- AI assistant and generated reports — tables modelled, no inference code
- Notifications delivery, real-time updates, file storage, reporting and export
- PostgreSQL row-level security as a second isolation layer
- Deployment configuration is not committed to the repository

---

Technology

Frontend — React 19, TypeScript, Vite, React Router, TanStack Query, Zustand, React Hook Form, Zod, Tailwind CSS, Radix UI, Recharts, Framer Motion, cmdk

Backend — Python 3.12, FastAPI, Pydantic v2, SQLAlchemy 2.0 (async), Alembic, passlib with Argon2, PyJWT, structlog

Data — PostgreSQL 16 via asyncpg, Redis 7 for rate-limit counters

Security — Argon2 hashing, JWT with algorithm pinning and type validation, server-side session records with rotation and reuse detection, RBAC, CORS allowlisting, security headers, trusted-proxy IP resolution, rate limiting, audit logging, log redaction, startup configuration validation

Infrastructure — Docker and Docker Compose, nginx for static serving, GitHub Actions CI, structured logging with optional error tracking

Testing — pytest with async fixtures against live PostgreSQL and Redis, Vitest with Testing Library, ruff, mypy

---

What This Project Demonstrates

NexusOS is evidence of platform engineering rather than feature assembly.

It shows multi-tenant SaaS architecture where the tenant boundary is a property of the system — derived server-side, enforced by both application and database constraints, and tested by attempting to cross it — rather than a filter each query is trusted to remember.

It shows security engineering as design rather than decoration: authorization re-derived from state so revocation is immediate, refresh-token reuse treated as a detectable compromise, a trusted-proxy boundary so IP-based controls cannot be spoofed, and configuration validated at startup so unsafe production settings fail the deploy. Equally, it shows the judgment to name what is absent — row-level security, issuer validation, account lockout — instead of implying completeness.

It shows full-stack depth: a normalised multi-tenant schema with a deliberate constraint strategy, a layered async API with a uniform contract, and a frontend built as a token and component system with a correctly handled token-refresh lifecycle.

And it shows production-minded sequencing. The hard, unglamorous layer was built first, tested where it matters, and documented honestly — including the boundary between what runs today and what is only prepared for.

Architecture artifacts: System Architecture · Security Architecture · Multi-Tenant Request Flow. All technical claims in this document were verified against the implemented codebase.